

# CONFIGURING LOCAL AI AND WEB SERVER

## 2025



**PREPARED BY**  
Nathan Regan

**CCNA**  
Period 0 - 2

## **Lab Purpose:**

The point of this lab is to run local LLM instances on our workstations. Doing so eliminates many potential security concerns that come with using online models like chat GPT, as they potentially have access to all the data given to their models.

## **Background:**

The history of ai starts with the theories of how neurons might work in the human brain, and how they may be simulated by a computer. The idea that neurons could be represented by electrical components dates back to the 1940's when a neurophysiologist named Warren McCulloch and mathematician Walter Pitts worked together to represent a model for a way that neurons could work in the human mind.

The first types of neural networks would consist of a single layer of neurons that would be trained by a linear regression. After this came the perceptron, a mechanical attempt to simulate the way that visual information is processed by the human mind. One version consisted of an array of 400 photocells that connected to a hidden layer of 512 perceptrons, which would pass onto the output layer 8 8 perceptrons. The connections between the layers would be random, and the weights would be adjusted between the output layer and their hidden layer. This technology was one of the first attempts at trying to classify images using computers.

As time went on, there were more and more discoveries and innovations in the ai research sphere, but each of these programs were very narrow in their scope. AI could be trained to classify images extremely well, or to predict the next pixel in an image, or how to play chess, but that program would only be able to do that one thing at a superhuman level. That is where a modern development in the ai research space has changed things. In a landmark research paper called attention is all you need, a group of researchers created a new type of ai model that used transformers to generate english text. This approach has led to an explosion in the ai industry, leading to trillions of dollars of investment into these models.

The transformer model has transformed artificial intelligence from a narrow technology where each model could only do one thing, to a general purpose tool capable of tackling a wide range of problems. The implications are massive, as these ai models are able to do many tasks previously thought to be non-automatable, like writing reports, writing code, or most other intellectual tasks.

The widespread use of ai tools like chat GPT comes with troubling security concerns, as its extremely broad usage could lead to scores of sensitive information being entered into online ai , like proprietary design documents, sensitive code, and medical documents. This data can and is used to train these ai models, and this means that a malicious party can gain access to data they are not supposed to.

As these LLMs have been developed, there have also been several models that have been open sourced and released publicly. Doing so allows users to have complete control over all data inputted into the LLMs, and tune them for their own purposes.

## **Lab Summary:**

In this lab, I will install and run Gemma 4b by running open web ui using docker. I will also need to install proper NVIDIA drivers in order to make sure my GPU is utilized on my Ubuntu machine

## Lab Config and Commands:

```
#Updates package lists
sudo apt-get update
```

```
#Installs packages necessary to add docker repository to Apt
sources
sudo apt-get install ca-certificates curl gnupg
```

```
# Assigns read, execute and write permissions to owner and assigns
# read and execute to other users inside the user group and all
# users in the designated directory
sudo install -m 0755 -d /etc/apt/keyrings
```

```
# Download and sign the GPG key for docker
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg
--dearmor -o /etc/apt/keyrings/docker.gpg
```

```
# Give read permissions to all users for the specified file
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

```
# Add the repository to Apt sources:
echo \
  "deb [arch=$(dpkg --print-architecture)
signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

```
#Updates package lists
sudo apt-get update
```

```
#Installs docker
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-
buildx-plugin docker-compose-plugin
```

```
#Creates shell variable containing the current distribution of
linux
distribution=$(. /etc/os-release;echo $ID$VERSION_ID)
```

```
#Download the nvidia-docker repository gpg key and add it to apt
curl -s -L https://nvidia.github.io/nvidia-docker/gpgkey | sudo
apt-key add -
```

```
#Adds the source list for nvidia-docker to apt
curl -s -L
https://nvidia.github.io/nvidia-docker/$distribution/nvidia-
docker.list | sudo tee /etc/apt/sources.list.d/nvidia-docker.list
```

```
#Updates package list and installs nvidia-container-toolkit
sudo apt-get update && sudo apt-get install -y nvidia-container-
toolkit
```

```
#Restarts docker processes
sudo systemctl restart docker
```

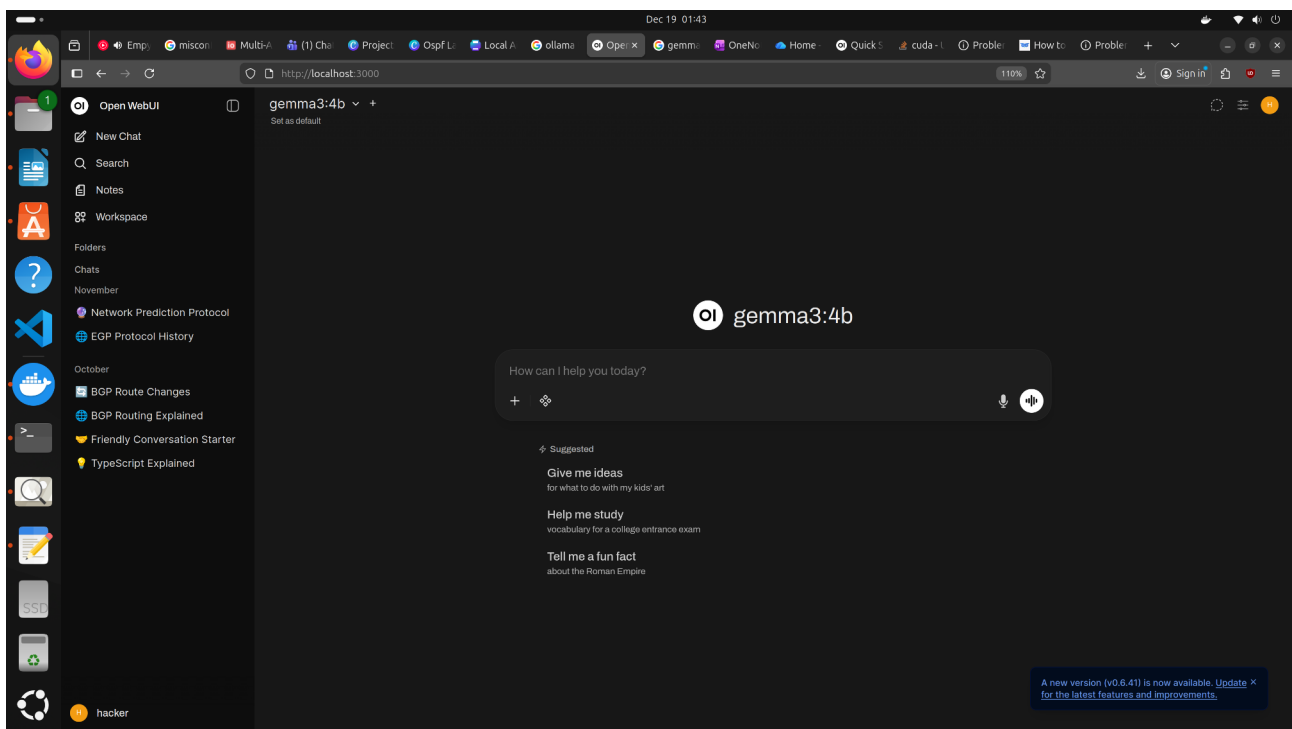
```
#Installs the docker image for open-webui
docker pull ghcr.io/open-webui/open-webui:main
```

```
#Runs open webui on port 4000 with gpu access
sudo docker run -d -p 4000:8080 --gpus all --add-
host=host.docker.internal:host-gateway -v
docker-open-webui:/app/backend/data --name docker-open-webui --
restart always ghcr.io/open-webui/open-webui:cuda
```

```
#Installs and runs the ollama docker image on port 11434
docker run -d --gpus=all -v ollama:/root/.ollama -p 11434:11434 --
name ollama ollama/ollama
```

```
#Tells the running ollama container to start running gemma3:4b
docker exec -it ollama ollama run gemma3:4b
```

Create an admin account, and simply ask the model what you need



The model should be accessible on the local network by typing in the devices ip address and using the port number 3000. Make sure to turn off the firewall of the device if connection is not working.

## Problems:

### Locked out of windows:

The first problem I encountered was that my windows had activated a 2fa check, requiring me to type in a code on my phone to recover access to my account. I had forgotten my phone that day, so I

decided to simply install flash Debian 13 on a separate computer and overwrite the windows image on the machine

**Debian 13 did not support NVIDIA container toolkit:**

After figuring out how to install the correct NVIDIA drivers for my Debian 13 system, I discovered that the NVIDIA container toolkit was not supported for Debian 13. This required me to flash a different operating system onto my machine, and I chose Ubuntu for its ease of access.

**Accidentally installed ARM drivers, had to reimage the machine:**

When Ubuntu was installed, I spent about an hour attempting to install ARM GPU drivers to support my NVIDIA GPU. After realizing the error and uninstalling the ARM drivers, I attempted to install the NVIDIA container toolkit and reinstall the correct NVIDIA graphics card drivers. However, I did not have the skill required to repair my instance, and had to reinstall Ubuntu to reset my drivers

**Docker would not use the graphics card:**

The issue that took the most amount of time was figuring out how to make the docker instance use the graphics card. It took hours of research and trial an error before I had to restart and reimage my instance. After reinstalling Ubuntu, I was able to find the correct commands that would allow docker to use the graphics card.